

Bonus 2 — Attacco a un database MySQL

Studente: Francesco Terracciano

Corso: Cyber Security & Ethical Hacking

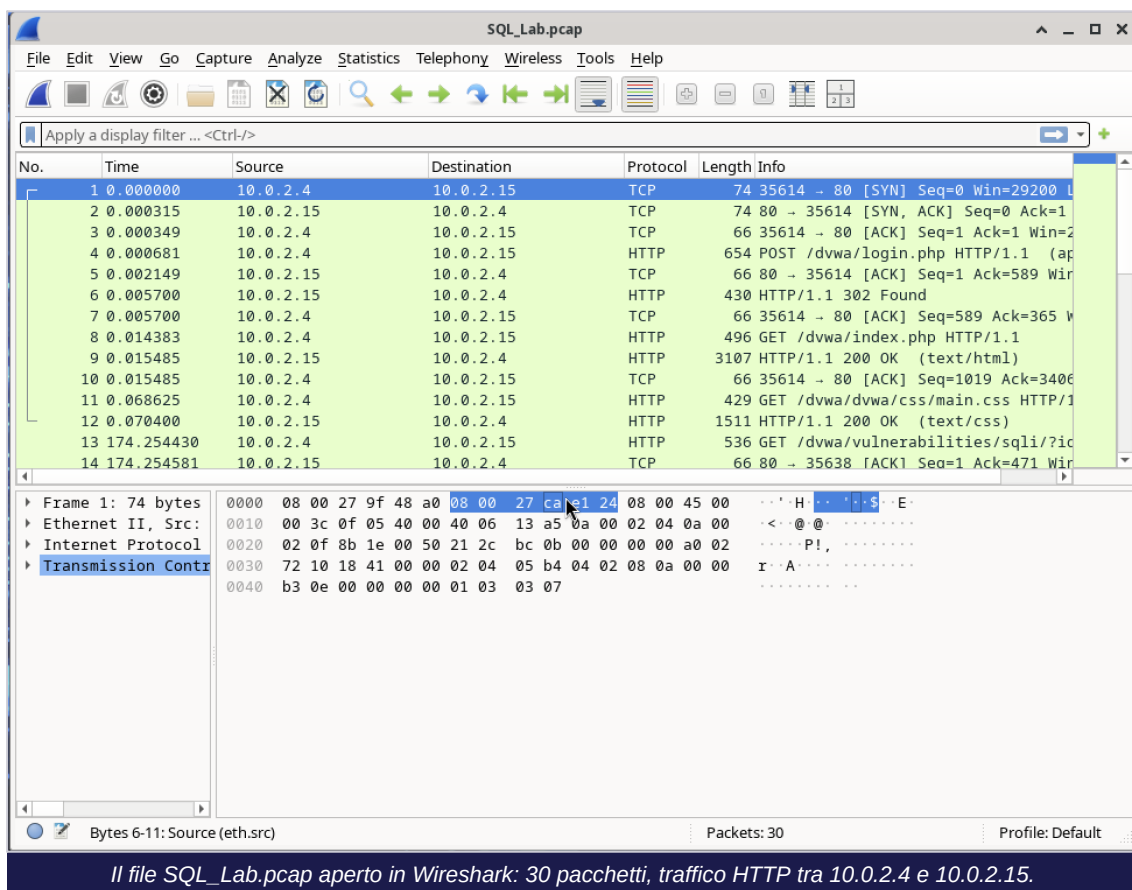
Data: 04/09/2026

Obiettivo e ambiente di lavoro

L'obiettivo del laboratorio è analizzare, tramite **Wireshark**, una cattura di rete (`SQL_Lab.pcap`) che documenta un attacco di **SQL injection** condotto in precedenza contro un database SQL. Seguendo il flusso applicativo pacchetto per pacchetto si ricostruiscono le fasi dell'attacco: dalla verifica della vulnerabilità fino all'estrazione degli hash delle password. L'attività è stata svolta sulla VM CyberOps Workstation; il file di cattura si estende per circa 8 minuti (441 secondi) e contiene 30 pacchetti.

Parte 1 — Apertura del file PCAP in Wireshark

Aperta l'applicazione Wireshark, è stato caricato il file `SQL_Lab.pcap` dalla directory `/home/analyst/lab.support.files/`. La cattura mostra il traffico HTTP scambiato tra due host durante l'attacco.

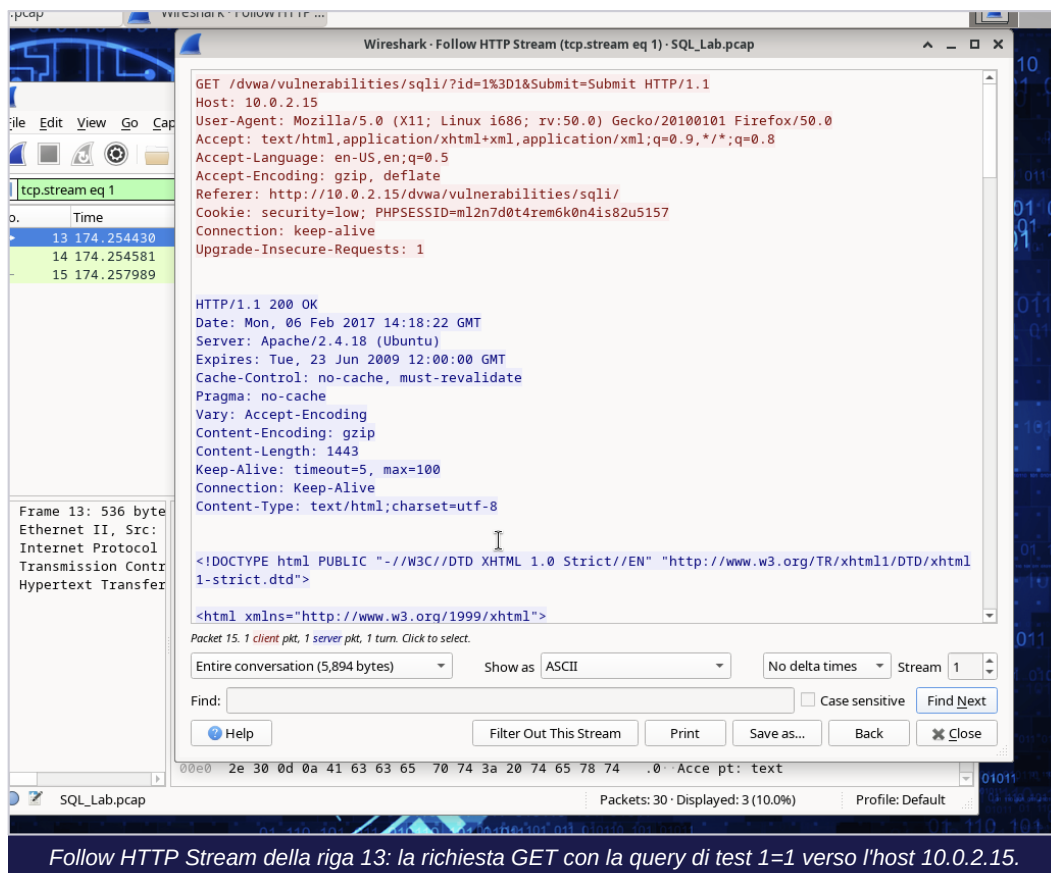


Il file `SQL_Lab.pcap` aperto in Wireshark: 30 pacchetti, traffico HTTP tra 10.0.2.4 e 10.0.2.15.

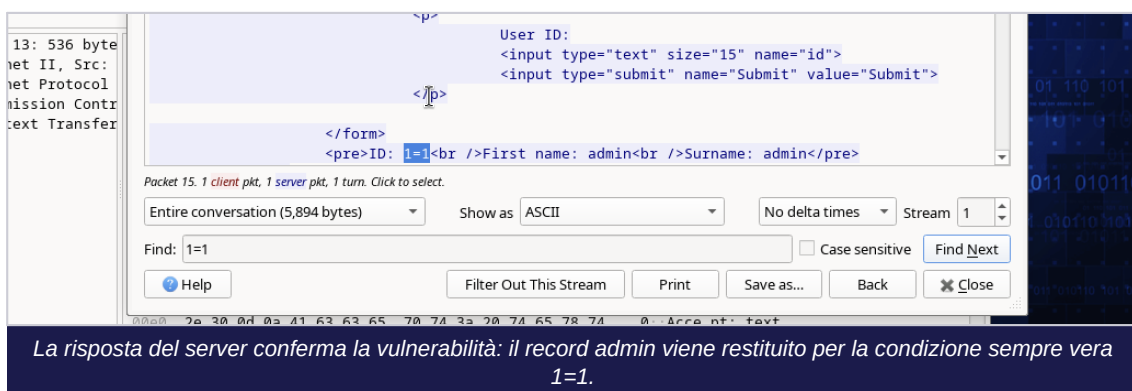
Quali sono i due indirizzi IP coinvolti nell'attacco? 10.0.2.4 (l'aggressore) e 10.0.2.15 (la vittima, che ospita l'applicazione web DVWA e il database).

Parte 2 — Inizio dell'attacco: test di vulnerabilità

Facendo clic destro sulla riga 13 (una richiesta GET HTTP) e scegliendo *Segui > Flusso HTTP*, si osserva la prima query inserita dall'aggressore nel campo UserID: **1'=1** (cioè **1=1**). In rosso la richiesta del client, in blu la risposta del server.

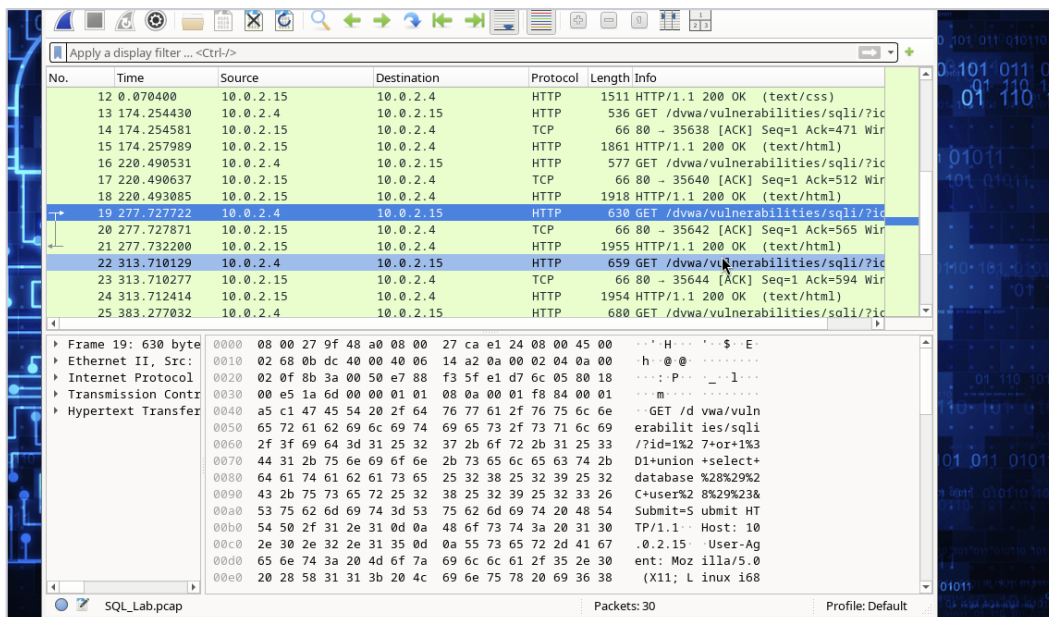


Invece di restituire un errore di login, l'applicazione risponde con un record del database (ID: 1=1 — First name: admin — Surname: admin). La stringa **1=1** crea una condizione SQL sempre vera: l'aggressore ha così verificato che l'applicazione è vulnerabile alla SQL injection, perché il database elabora e risponde all'input iniettato.

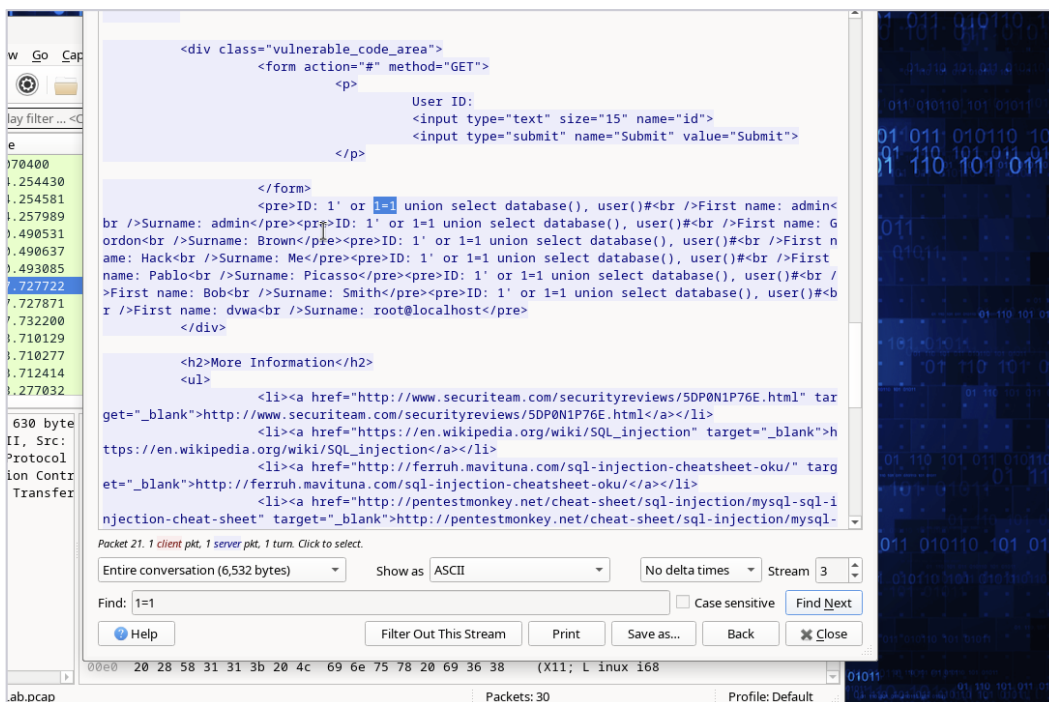


Parte 3 — L'attacco continua: nome del database e utente

Seguendo il flusso della riga 19, si vede una query più elaborata: **1' or 1=1 union select database(), user()#**. La tecnica UNION SELECT permette di aggiungere ai risultati legittimi i dati richiesti dall'aggressore.



Selezione della riga 19 nella cattura: nuova richiesta GET con query UNION SELECT.

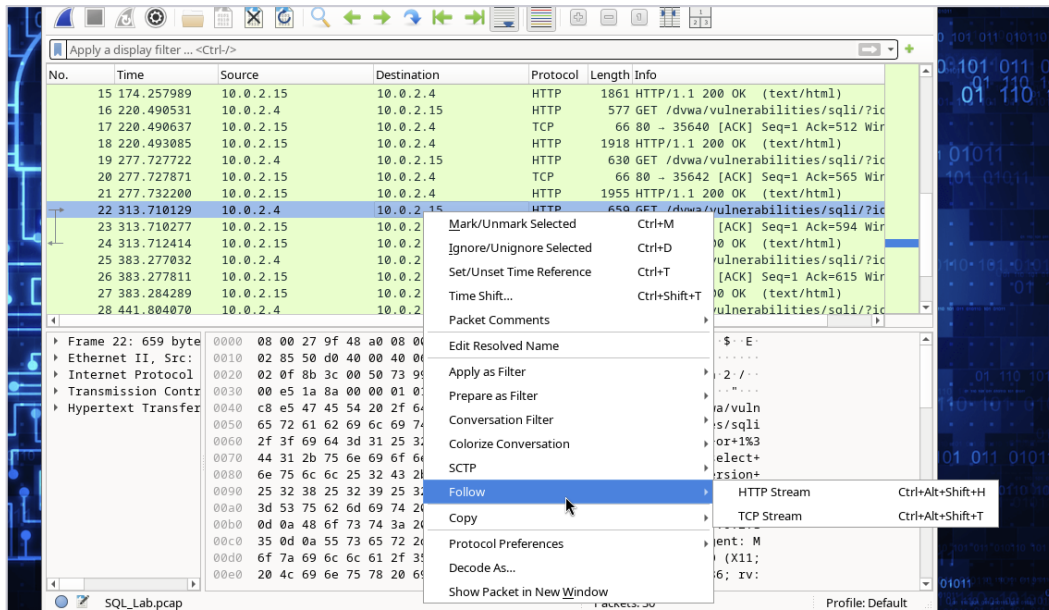


Follow HTTP Stream della riga 19: la risposta rivela il nome del database e l'utente, oltre agli account applicativi.

Nome del database e utente del database: Il database si chiama **dvwa** e l'utente del database è **root@localhost**. La risposta elenca anche i vari account applicativi (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith).

Parte 4 — Informazioni di sistema: la versione

L'aggressore prosegue mirando a informazioni più specifiche. Nel flusso della riga 22 compare la query **1' or 1=1 union select null, version ()#**, che punta a individuare la versione del database. L'identificatore di versione si trova alla fine dell'output, subito prima del codice HTML di chiusura.



Menu contestuale su una richiesta GET: Segui > Flusso HTTP per analizzare la query della versione.

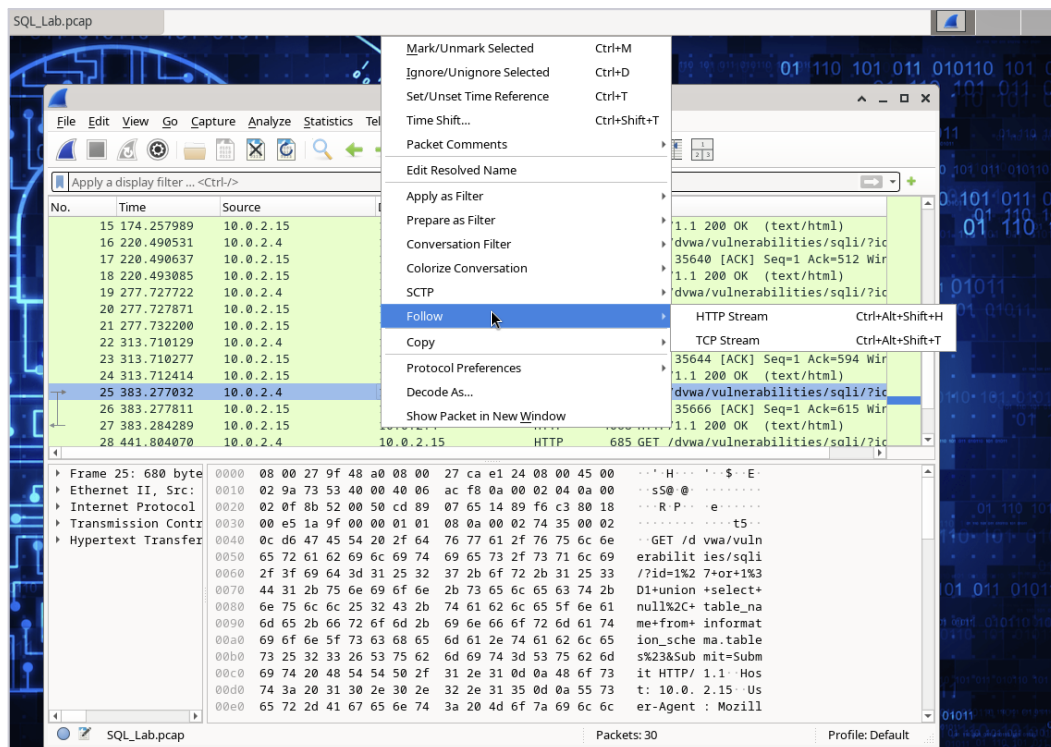
```
</form>
  <pre>ID: 1' or 1=1 union select null, version ()#<br />First name: admin<br />Surname: admin</pre><pre>ID: 1' or 1=1 union select null, version ()#<br />First name: Gordon<br />Surname: Brown</pre><pre>ID: 1' or 1=1 union select null, version ()#<br />First name: Bob<br />Surname: Me</pre><pre>ID: 1' or 1=1 union select null, version ()#<br />First name: Picasso<br />Surname: Picasso</pre><pre>ID: 1' or 1=1 union select null, version ()#<br />First name: Smith<br />Surname: Smith</pre><pre>ID: 1' or 1=1 union select null, version ()#<br />First name: Bob<br />Surname: 5.7.12-0ubuntu1.1</pre>
</div>
```

La risposta espone la versione del database MySQL alla fine dell'output.

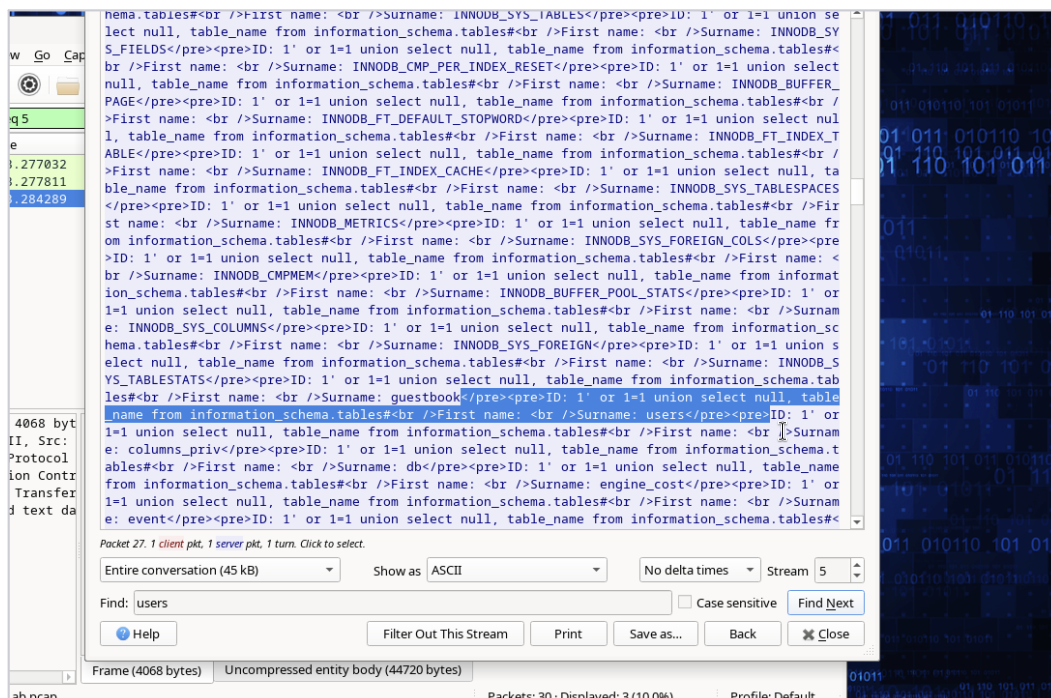
Qual è la versione? MySQL 5.7.12-0ubuntu1.1.

Parte 5 — Le informazioni sulle tabelle

Sapendo che il database contiene numerose tabelle, l'aggressore tenta di elencarle. Nel flusso della riga 25 la query è `1' or 1=1 union select null, table_name from information_schema.tables#`. Avendo specificato "null" senza ulteriori filtri, l'output è enorme e comprende tutte le tabelle.



Segui > Flusso HTTP sulla riga 25 per elencare tutte le tabelle del database.

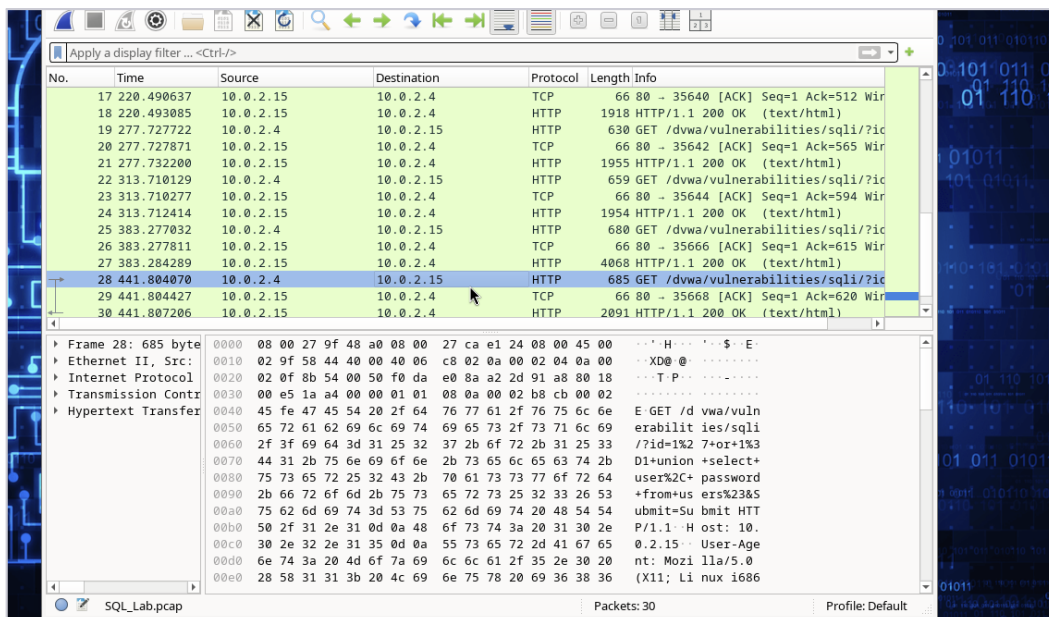


L'output elenca moltissime tabelle (INNODB_, guestbook, users, ...) di information_schema.*

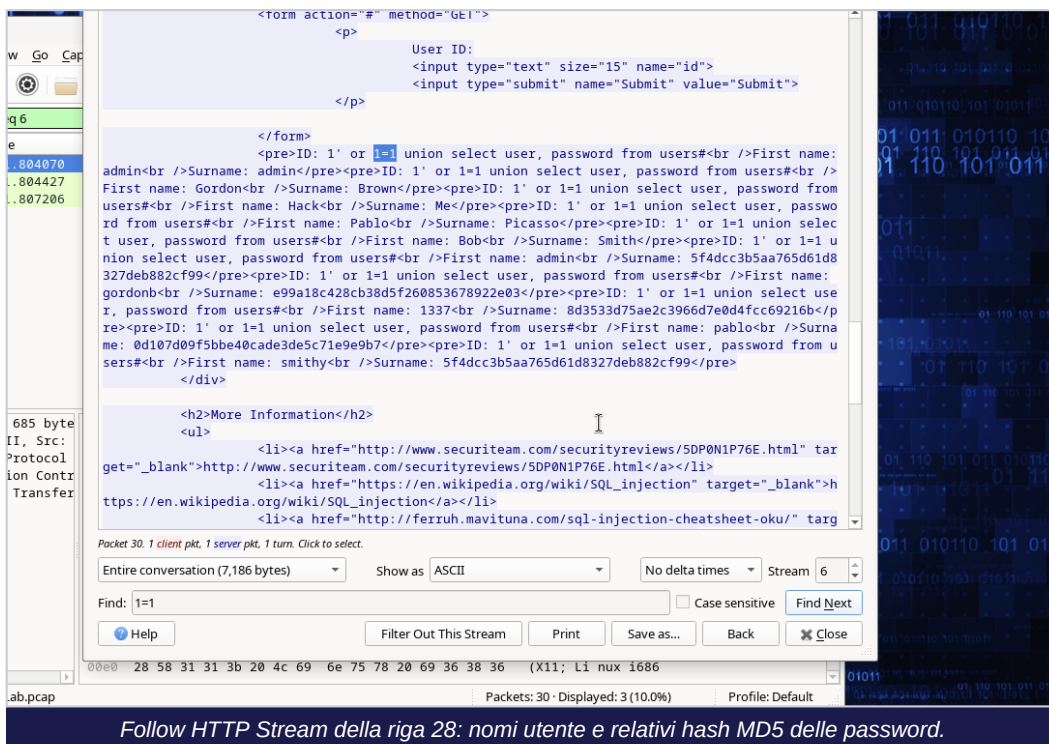
Cosa farebbe il comando modificato con `WHERE table_name='users'`? La query `1' OR 1=1 UNION SELECT null, column_name FROM INFORMATION_SCHEMA.columns WHERE table_name='users'` restituirebbe un output molto più breve, filtrato per elencare solo le colonne appartenenti alla tabella "users".

Parte 6 — La conclusione: gli hash delle password

L'attacco si conclude con il bottino più prezioso. Nel flusso della riga 28 la query è **1' or 1=1 union select user, password from users#**, che estrae direttamente nomi utente e hash delle password dalla tabella users.



Selezione della riga 28: la richiesta che estrae utenti e hash delle password.



Follow HTTP Stream della riga 28: nomi utente e relativi hash MD5 delle password.

Quale utente ha l'hash 8d3533d75ae2c3966d7e0d4fcc69216b? L'utente **1337**.

Copiando l'hash in un servizio di cracking online (crackstation.net), l'hash — di tipo **MD5** — viene decifrato immediatamente, a conferma di quanto sia debole una password memorizzata come semplice hash MD5 non salato.

Enter up to 20 non-salted hashes, one per line:

8d3533d75ae2c3966d7e0d4fcc69216b

☐ Non sono un robot 

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, rpeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin), QubesV3.1BackupDefaults

hashes: 1	cracked: 1	partial: 0	not found: 0	invalid: 0
Hash	Type	Result		
8d3533d75ae2c3966d7e0d4fcc69216b	md5	charley		

Color Codes: **Green:** Exact match, **Yellow:** Partial match — the hash column shows what that word really hashes to, with the **part yours matched** highlighted, **Red:** Not found.

CrackStation decifra l'hash MD5 8d3533... rivelando la password in chiaro.

Qual è la password in chiaro? **charley** (l'hash 8d3533d75ae2c3966d7e0d4fcc69216b è un MD5 non salato).

Domande di Riflessione

1. Qual è il rischio che le piattaforme utilizzino il linguaggio SQL?

I siti web sono comunemente basati su database e utilizzano il linguaggio SQL per interrogarli. Se l'input dell'utente non viene adeguatamente validato, un aggressore può iniettare istruzioni SQL e far eseguire al database comandi non previsti. La gravità dipende dall'aggressore e da cosa riesce a ottenere: come mostra questo laboratorio, si passa dalla semplice verifica della vulnerabilità fino all'esfiltrazione completa di struttura del database, credenziali e hash delle password.

2. Quali sono almeno due metodi per prevenire gli attacchi di SQL injection?

Le contromisure principali includono: **validare e filtrare l'input dell'utente** (non fidarsi mai dei dati in ingresso); utilizzare **query parametrizzate / prepared statement** anziché concatenare direttamente l'input nelle stringhe SQL; adottare **parametri con le stored procedure**; implementare un **Web Application Firewall (WAF)**; disabilitare funzionalità non necessarie del database e limitare i privilegi dell'account applicativo; e monitorare le istruzioni SQL eseguite. Inoltre, sul fronte credenziali, le password non andrebbero mai salvate come semplici hash MD5 non salati, ma con algoritmi robusti e "salt".

Sintesi. La cattura documenta un attacco di SQL injection completo contro l'applicazione DVWA (vittima 10.0.2.15) condotto da 10.0.2.4. Partendo dal test `1=1`, l'aggressore ha progressivamente estratto nome del database (dvwa) e utente (root@localhost), la versione (MySQL 5.7.12), l'elenco delle tabelle e infine username e hash delle password, uno dei quali (utente 1337) è stato decifrato in chiaro come "charley". Il caso evidenzia l'importanza della validazione dell'input, delle query parametrizzate e di una corretta gestione delle credenziali.